

INSTITUTO
FEDERAL
Santa Catarina

Resumo de matérias

Redes de Computadores

Eduardo Cardoso Oliveira, 2025

Sumário

1 Como estudar para esta disciplina? 4

2 Capítulo 1 — Introdução 5
2.1 O que é a Internet? . . . 5
2.2 Protocolos . . . 5
2.3 Tamanho das redes: LAN, MAN e WAN . . . 5
2.4 Borda da rede: hosts e redes de acesso . . . 5
2.5 Mídia física . . . 6
2.6 O núcleo da rede: comutação de pacotes x comutação de circuitos . . . 6
2.7 Atraso, perda e taxa de transferência . . . 7
2.8 Segurança de rede . . . 7
2.9 Camadas de protocolo e encapsulamento . . . 7

3 Capítulo 2 — Camada de Aplicação 9
3.1 Paradigmas de arquitetura: cliente-servidor x P2P . . . 9
3.2 Processos, sockets e requisitos de transporte . . . 9
3.3 Web e HTTP . . . 9
3.4 E-mail: SMTP e IMAP . . . 10
3.5 DNS: Sistema de Nomes de Domínio . . . 11

4 Capítulo 3 — Camada de Transporte 12
4.1 Multiplexação e demultiplexação . . . 12
4.2 UDP: o protocolo simples . . . 12
4.3 Transferência confiável de dados (rdt): a ideia por trás do TCP . . . 12
4.4 TCP: transporte confiável orientado à conexão . . . 13
4.5 Controle de congestionamento TCP . . . 13

5 Capítulo 4 — Camada de Rede: Plano de Dados 14
5.1 Formato do datagrama IP . . . 14
5.2 Endereçamento IP, sub-redes e CIDR . . . 14
5.3 NAT: tradução de endereços de rede . . . 14
5.4 IPv6 . . . 15

6 Capítulo 5 — Camada de Rede: Plano de Controle 16
6.1 Algoritmos de roteamento . . . 16
6.1.1 Algoritmo de Dijkstra: exemplo resolvido . . . 16
6.2 Roteamento na Internet real: OSPF e BGP . . . 17
6.3 Plano de controle SDN . . . 17

7 Capítulo 6 — A Camada de Enlace 18
7.1 Serviços da camada de enlace . . . 18
7.2 Detecção de erros: paridade e CRC . . . 18
7.3 Protocolos de acesso múltiplo . . . 18
7.4 Endereçamento: MAC e ARP . . . 19
7.5 Ethernet . . . 19
7.6 Switches: encaminhamento e autoaprendizagem . . . 19

8 Referências 20

Aviso importante e Isenção de responsabilidade

1. **Finalidade do documento**
   Este documento foi elaborado com o objetivo de servir como um material de apoio e revisão para os estudos das matérias do segundo semestre de Análise e Desenvolvimento de Sistemas. Todos os conteúdos aqui presentes foram retirados de anotações pessoais e resumos, compilados para facilitar a compreensão do leitor.

2. **Fontes e Direitos Autorais**
   Os resumos e anotações aqui presentes foram desenvolvidos a partir dos materiais de sala disponibilizados pelos professores (como slides, apostilas, listas de exercícios, livros, etc), anotações de aula e pesquisas. Os direitos autorais sobre os materiais originais (aulas, livros, artigos) permanecem integralmente aos seus respectivos autores e editoras. Este documento é uma obra derivada, de caráter estudantil e sem fins lucrativos.

3. **Precisão das Informações**
   Desde já, o autor não oferece nenhuma garantia de que os conteúdos aqui contidos sejam precisos e corretos, por mais que todo esforço tenha sido feito para garantir que tais informações sejam corretas. Erros, omissões ou interpretações equivocadas podem existir. O autor não se responsabiliza por quaisquer consequências, acadêmicas ou de outra natureza, que possam surgir do uso deste material.

4. **Uso Recomendado**
   Este material **NÃO SUBSTITUI DE FORMA ALGUMA** a leitura de bibliografias recomendadas por professores, presença em aulas, consulta às fontes oficiais e, principalmente, as orientações do corpo docente. Utilize este documento como uma ferramenta complementar aos seus estudos.

5. **Vínculo Institucional**
   Este é um projeto independente e não possui qualquer vínculo oficial com o Instituto Federal de Santa Catarina Campus São José, a Coordenação do Curso de Análise e Desenvolvimento de Sistemas, ou os professores das disciplinas. Não há qualquer endosso ou revisão oficial do conteúdo por parte da instituição.

Apresentação do Material

Olá, colega!

Seja bem-vindo(a) ao meu resumo de estudos! Este documento foi criado ao longo do semestre com o objetivo de centralizar e facilitar o acesso aos conteúdos que foram abordados durante as aulas do segundo semestre do curso em 2025.2 na disciplina de Redes de Computadores.

O que você vai encontrar neste documento?

- **Resumos:** Para esta disciplina trago resumo das aulas e dos slides (baseados principalmente no livro do Kurose & Ross), levantando pontos que considero os mais importantes de cada camada da pilha de protocolos.
- **Anotações e comentários:** Anotações sobre conceitos mais complicados e comentários sobre a matéria, sempre tentando explicar com minhas próprias palavras.
- **Exemplos resolvidos:** Como essa disciplina tem bastante cálculo (atraso, subredes, CRC, algoritmos de roteamento), trago exemplos numéricos resolvidos passo a passo.
- **Documentos e bibliografias:** Quando possível, vou disponibilizar materiais, resumos, listas de exercícios, gabaritos e referências bibliográficas para auxiliar.

Como este documento está organizado?

O material foi dividido por capítulos, seguindo a mesma ordem do livro-texto da disciplina (Kurose & Ross, "Redes de Computadores: Uma Abordagem Top-Down") e do andamento do semestre 2025.2: começamos com uma visão geral da Internet e vamos "descendo"pela pilha de protocolos, camada por camada.

Informações adicionais e contato

Este material foi feito de aluno para aluno; sugestões e colaborações são sempre bem-vindas! Se encontrar algum erro, tiver alguma sugestão para adicionar ao documento ou tirar alguma dúvida sobre qualquer matéria, fique à vontade para entrar em contato.

- **Autor:** Eduardo Cardoso Oliveira
- **E-mail Institucional:** eduardo.co22@aluno.ifsc.edu.br
- **Github:** https://github.com/EduCardoso22

1 Como estudar para esta disciplina?

Redes de Computadores é uma daquelas matérias que parecem abstratas no começo (afinal, não dá para "ver"um pacote de dados passando pelo cabo), mas que na verdade segue uma lógica bem organizada: a disciplina inteira é estruturada em torno da ideia de **camadas**, e cada capítulo do livro corresponde a uma camada da pilha de protocolos da Internet. Meu principal conselho é: sempre que for estudar um assunto novo, pare e se pergunte "em qual camada eu estou?"e "o que essa camada resolve que a camada de baixo não resolve sozinha?". Isso ajuda muito a não misturar conceitos de camadas diferentes (por exemplo, confundir um endereço IP, da camada de rede, com um endereço MAC, da camada de enlace).

Outro ponto importante é que boa parte da disciplina envolve cálculos (atraso de transmissão e propagação, cálculo de sub-redes, checksum, CRC, algoritmos de roteamento como Dijkstra). Esses cálculos seguem fórmulas e procedimentos bem definidos, então vale muito a pena **praticar exercícios numéricos** repetidamente até o processo ficar automático, exatamente como se faz em matemática. Por fim, use e abuse de ferramentas visuais: o Wireshark (para capturar pacotes reais) e simuladores de rede ajudam demais a tornar esses conceitos abstratos em algo concreto que você realmente vê acontecendo.

2 Capítulo 1 — Introdução

2.1 O que é a Internet?

Podemos descrever a Internet de duas formas complementares. Pela ótica dos **componentes físicos**, a Internet é uma rede de bilhões de dispositivos conectados (chamados de **hosts** ou **sistemas finais**), interligados por **comutadores de pacotes** (roteadores e switches) através de **enlaces de comunicação** (fibra, cobre, rádio, satélite), cuja capacidade de transmissão é medida em **largura de banda**. Essa "rede de redes"é formada por diversos Provedores de Serviço de Internet (ISPs) interconectados entre si. Pela ótica de **serviços**, a Internet é uma infraestrutura que fornece serviços a aplicações distribuídas (Web, streaming de vídeo, e-mail, etc), oferecendo uma interface de programação (os **sockets**) para que essas aplicações possam "se conectar"e usar o serviço de transporte da rede.

2.2 Protocolos

Um **protocolo** é o conjunto de regras que define o formato e a ordem das mensagens trocadas entre as entidades de uma rede, assim como as ações tomadas na transmissão e no recebimento dessas mensagens. É útil pensar em analogia com protocolos humanos: quando alguém diz "Oi, que horas são?"e espera uma resposta, existe uma "regra social"implícita de pergunta/resposta. Em uma rede de computadores, isso é formalizado: por exemplo, antes de pedir uma página web, o navegador abre uma conexão TCP (um "oi") e só depois envia a solicitação `GET`.

2.3 Tamanho das redes: LAN, MAN e WAN

Um ponto conceitual simples, mas importante, é a classificação das redes pelo seu alcance geográfico:

- **LAN (Local Area Network)**: rede de área local, geralmente dentro de um prédio, escritório, campus ou residência.
- **MAN (Metropolitan Area Network)**: escopo intermediário entre uma LAN e uma WAN, cobrindo uma área geográfica de tamanho médio, como um município ou região metropolitana.
- **WAN (Wide Area Network)**: rede de área ampla, cobrindo distâncias relativamente (ou muito) longas, frequentemente conectando LANs distantes entre si.

2.4 Borda da rede: hosts e redes de acesso

A **borda da rede** é onde ficam os hosts (clientes e servidores, geralmente em data centers) e as **redes de acesso**, que são os links que conectam esses sistemas finais ao primeiro roteador (o roteador de borda). Os principais tipos de acesso trabalhados foram:

- **DSL (Digital Subscriber Line)**: usa a linha telefônica existente, separando a transmissão de voz e dados em frequências diferentes. Taxas típicas: 24 a 52 Mbps no *downstream* e 3,5 a 16 Mbps no *upstream*.

- **Acesso a cabo (HFC)**: usa uma rede híbrida de fibra e coaxial (Hybrid Fiber-Coaxial), compartilhada entre as residências de uma mesma região, com taxas assimétricas (downstream de até 40 Mbps a 1,2 Gbps, upstream de 30 a 100 Mbps).
- **FTTH (Fiber-to-the-Home)**: leva um caminho de fibra ótica diretamente até a residência, geralmente oferecendo as maiores taxas de transmissão.
- **Redes sem fio**: podem ser locais (WLANs, como o WiFi, cobrindo cerca de 70 metros) ou de área ampla (redes celulares 4G/5G, fornecidas pela operadora móvel, cobrindo quilômetros).

2.5 Mídia física

Os links físicos podem usar **mídia guiada** (os sinais se propagam em meios sólidos, como par trançado, cabo coaxial e fibra óptica) ou **mídia não guiada** (os sinais se propagam livremente pelo ar, como no rádio). A fibra óptica se destaca por permitir transmissão de altíssima velocidade (10 a 100 Gbps) com baixíssima taxa de erro, por ser imune a interferências eletromagnéticas.

2.6 O núcleo da rede: comutação de pacotes x comutação de circuitos

O núcleo da rede é a malha de roteadores interconectados que tem duas funções essenciais: **encaminhamento** (uma ação *local* de cada roteador, que move um pacote do link de entrada para o link de saída correto, consultando sua tabela de encaminhamento) e **roteamento** (uma ação *global*, executada pelos algoritmos de roteamento, que determina os caminhos de origem até destino usados pelos pacotes).

Existem dois grandes paradigmas de transferência de dados:

- **Comutação de pacotes**: os hosts dividem as mensagens em pacotes menores, que são enviados de forma "armazena-e-reenvia"(*store-and-forward*): o pacote inteiro precisa chegar a um roteador antes de ser retransmitido ao próximo link. Como vários fluxos de dados compartilham o mesmo link, pode ocorrer **enfileiramento** (quando a taxa de chegada de dados excede temporariamente a capacidade do link de saída) e, em casos extremos, **perda de pacotes** (quando o buffer do roteador fica cheio).
- **Comutação de circuitos**: recursos são reservados de ponta a ponta (fim-a-fim) para uma "chamada"antes mesmo dos dados começarem a ser enviados, como acontece na rede telefônica tradicional. O desempenho é garantido (semelhante a ter um circuito dedicado), mas o recurso fica ocioso se não estiver sendo usado por aquela chamada (não há compartilhamento).

A comutação de pacotes é mais eficiente para tráfego "intermitente"(que às vezes tem dados para enviar, às vezes não), porque permite compartilhar recursos entre vários usuários — mas isso tem como custo a possibilidade de congestionamento, atraso e perda de pacotes, exigindo protocolos para transferência confiável de dados e controle de congestionamento (que veremos no Capítulo 3, com o TCP).

Exemplo numérico de atraso de transmissão em um salto: se um pacote tem $L = 10$ Kbits e o link tem taxa de transmissão $R = 100$ Mbps, o atraso de transmissão (tempo para "empurrar"o pacote inteiro para o link) é:

$$d_{trans} = \frac{L}{R} = \frac{10.000 \text{ bits}}{100.000.000 \text{ bps}} = 0,1 \text{ ms}$$

2.7 Atraso, perda e taxa de transferência

O atraso total que um pacote sofre em um nó da rede (o **atraso nodal**, d_{nodal}) é composto por quatro fontes:

$$d_{nodal} = d_{proc} + d_{fila} + d_{trans} + d_{prop}$$

- d_{proc} (processamento nodal): tempo para verificar erros de bit e determinar o link de saída. Geralmente é da ordem de microssegundos.
- d_{fila} (atraso na fila): tempo de espera no buffer do link de saída, antes da transmissão. Depende do nível de congestionamento do roteador.
- d_{trans} (atraso de transmissão): $d_{trans} = L/R$, onde L é o tamanho do pacote (bits) e R é a taxa do link (bps).
- d_{prop} (atraso de propagação): $d_{prop} = d/s$, onde d é a distância física do link e s é a velocidade de propagação do sinal (aproximadamente 2×10^8 m/s).

É importante não confundir d_{trans} (depende do tamanho do pacote e da taxa do link) com d_{prop} (depende da distância física), são conceitos fisicamente diferentes, mesmo que ambos sejam medidos em tempo.

A **perda de pacotes** ocorre quando o buffer de um roteador está cheio e um novo pacote que chega é descartado. A **taxa de transferência** (*throughput*) é a taxa (em bits por unidade de tempo) na qual os bits de fato são entregues do remetente ao receptor; quando existem vários links com taxas diferentes no caminho, a taxa de transferência final é limitada pelo **link de gargalo** (o link mais lento do caminho).

2.8 Segurança de rede

A Internet não foi originalmente projetada pensando em segurança (a visão original era de um grupo pequeno de usuários que confiavam mutuamente). Isso explica por que hoje precisamos lidar com diversos tipos de ataque: **sniffing de pacotes** (interceptar e ler pacotes que passam por uma mídia compartilhada), **IP spoofing** (injetar pacotes com endereço de origem falso, se passando por outra máquina) e **negação de serviço (DoS)** (sobrecarregar um recurso com tráfego falso, tornando-o indisponível para o tráfego legítimo). As principais linhas de defesa incluem autenticação, confidencialidade (por criptografia), verificações de integridade (assinaturas digitais) e firewalls.

2.9 Camadas de protocolo e encapsulamento

Assim como o sistema de viagens aéreas é organizado em "camadas"de serviço (bilhete, bagagem, portão, pista, cada uma dependendo da camada abaixo), a pilha de protocolos da Internet é organizada em cinco camadas:

1. **Aplicação**: suporte às aplicações de rede (HTTP, SMTP, DNS).
2. **Transporte**: transferência de dados processo-a-processo (TCP, UDP).
3. **Rede**: roteamento de datagramas da origem ao destino (IP, protocolos de roteamento).
4. **Enlace**: transferência de dados entre elementos de rede vizinhos (Ethernet, WiFi/802.11).
5. **Física**: transmissão dos bits "no fio".

O uso de camadas facilita o design e a manutenção de sistemas complexos: uma alteração na implementação de uma camada é transparente para o restante do sistema, desde que o serviço oferecido continue o mesmo.

O processo de passar os dados de uma camada para a outra é chamado de **encapsulamento**: cada camada recebe os dados da camada de cima e adiciona o seu próprio cabeçalho, formando uma estrutura parecida com bonecas russas (Matryoshka) empilhadas. Assim, a mensagem (**M**) da camada de aplicação vira um **segmento** (com cabeçalho de transporte H_t) na camada de transporte, que vira um **datagrama** (com cabeçalho de rede H_n) na camada de rede, que por sua vez vira um **quadro** (com cabeçalho de enlace H_l) na camada de enlace. Cada roteador no caminho "desembrulha"o pacote até a camada de rede (olha o cabeçalho H_n para decidir o encaminhamento) e depois o "reembrulha"para o próximo link.

3 Capítulo 2 — Camada de Aplicação

3.1 Paradigmas de arquitetura: cliente-servidor x P2P

- **Cliente-servidor**: o servidor é um host sempre ativo, com endereço IP permanente, geralmente em um data center. Os clientes contatam e se comunicam com o servidor, podem estar conectados de forma intermitente e geralmente não se comunicam diretamente uns com os outros. Exemplos: HTTP, IMAP, FTP.

- **Ponto a ponto (P2P)**: não existe um servidor sempre ativo; os próprios sistemas finais (pares, ou *peers*) se comunicam diretamente entre si, solicitando e fornecendo serviços uns aos outros. Essa arquitetura é autoescalável (novos pares trazem tanto nova demanda quanto nova capacidade de serviço), mas o gerenciamento é mais complexo (os pares se conectam de forma intermitente e mudam de endereço IP). Exemplo: compartilhamento de arquivos P2P (BitTorrent).

3.2 Processos, sockets e requisitos de transporte

Um **processo** é um programa em execução em um host; dois processos em hosts diferentes se comunicam trocando mensagens através de um **socket**, que funciona como uma "porta": o processo empurra a mensagem para fora do socket e confia na infraestrutura de transporte da rede para entregá-la ao socket do processo que vai receber.

Diferentes aplicações têm necessidades diferentes quanto ao serviço de transporte:

- **Integridade dos dados**: algumas aplicações (transferência de arquivos, páginas web) exigem transferência 100% confiável; outras (áudio/vídeo em tempo real) toleram alguma perda.

- **Vazão (throughput)**: aplicações multimídia exigem uma vazão mínima para funcionarem bem; outras ("elásticas") se adaptam à taxa que conseguirem.

- **Tempo**: aplicações como telefonia pela Internet e jogos interativos exigem baixo atraso (sensibilidade ao tempo).

Para atender a essas necessidades, a Internet oferece dois serviços de transporte principais:

- **TCP**: orientado à conexão (exige uma configuração prévia entre cliente e servidor), oferece transporte confiável, controle de fluxo (o remetente não sobrecarrega o receptor) e controle de congestionamento (o remetente se ajusta quando a rede está sobrecarregada).

- **UDP**: não oferece confiabilidade, controle de fluxo nem controle de congestionamento, mas também não exige configuração de conexão — é mais "leve"e rápido, usado quando um pouco de perda é tolerável ou quando a própria aplicação implementa sua confiabilidade.

3.3 Web e HTTP

O **HTTP** (HyperText Transfer Protocol) é o protocolo de transferência de hipertexto, seguindo o modelo cliente/servidor: o navegador (cliente) solicita, recebe e exibe objetos

da Web (arquivos HTML, imagens, etc), enquanto o servidor envia os objetos em resposta às solicitações. O HTTP roda sobre TCP, usando a porta 80, e é um protocolo **sem estado** (*stateless*): o servidor não guarda informações sobre solicitações anteriores do cliente, o que mantém o protocolo mais simples (manter estado traria complicações, como reconciliar informações caso o servidor ou cliente falhem).

Existem dois tipos de conexão HTTP:

- **Não persistente**: cada objeto exige sua própria conexão TCP (abrir, enviar um objeto, fechar). O tempo de resposta para cada objeto é de $2 \times RTT$ + tempo de transmissão do arquivo (um RTT para abrir a conexão TCP, outro para a solicitação/resposta HTTP).
- **Persistente (HTTP 1.1)**: a conexão TCP fica aberta depois da resposta, permitindo que várias mensagens sejam trocadas na mesma conexão, reduzindo bastante o tempo de resposta (apenas um RTT para todos os objetos referenciados).

As principais mensagens HTTP são:

- **GET**: usado para solicitar um objeto, podendo incluir dados na própria URL (depois de um "?").
- **POST**: usado para enviar dados do cliente ao servidor (por exemplo, dados de um formulário) no corpo da mensagem.
- **HEAD**: solicita apenas os cabeçalhos que seriam retornados por um GET, sem o corpo.
- **PUT**: faz o upload de um arquivo, substituindo o conteúdo existente na URL especificada.

As respostas HTTP trazem um **código de status**, sendo os mais comuns: `200 OK` (sucesso), `301 Movido permanentemente`, `400 Solicitação incorreta`, `404 Não encontrado` e `505 Versão HTTP não suportada`.

Para manter algum "estado"do usuário mesmo sendo um protocolo sem estado, o HTTP usa **cookies**, que funcionam com quatro componentes: a linha de cabeçalho de cookie na resposta HTTP, a linha de cabeçalho de cookie na próxima solicitação, um arquivo de cookie guardado no navegador do usuário e um banco de dados no backend do site. Assim, o site consegue reconhecer o usuário em visitas e solicitações subsequentes.

Outro recurso importante é o **cache web** (proxy): atua como cliente e servidor ao mesmo tempo, guardando cópias de objetos recentemente solicitados mais perto do cliente, o que reduz o tempo de resposta e o tráfego no link de acesso da instituição.

3.4 E-mail: SMTP e IMAP

O sistema de e-mail é composto por três componentes principais: os **agentes de usuário** (o "leitor de e-mail", como o Outlook), os **servidores de correio eletrônico** (que mantêm a caixa postal do usuário e uma fila de mensagens de saída) e o protocolo **SMTP** (Simple Mail Transfer Protocol), que transfere as mensagens entre servidores.

Um cenário típico (Alice envia um e-mail para Bob) segue estes passos: (1) Alice redige a mensagem no seu agente de usuário; (2) o agente de Alice envia a mensagem ao servidor de e-mail dela via SMTP; (3) o servidor de Alice abre uma conexão TCP com o

servidor de e-mail de Bob; (4) a mensagem é enviada por essa conexão; (5) o servidor de Bob coloca a mensagem na caixa postal dele; (6) Bob usa seu agente de usuário para ler a mensagem.

É importante notar que o SMTP só cuida do **envio e entrega** da mensagem entre servidores, ele não é usado para o usuário final *buscar* suas mensagens no servidor. Para isso, existem protocolos de acesso a correio eletrônico, sendo o mais importante o **IMAP** (Internet Mail Access Protocol), que permite recuperar, excluir e organizar mensagens em pastas diretamente no servidor (diferente do antigo POP3, que baixava e removia as mensagens do servidor).

3.5 DNS: Sistema de Nomes de Domínio

Enquanto os roteadores da Internet usam **endereços IP** (números de 32 bits) para encaminhar datagramas, as pessoas preferem usar nomes mais "amigáveis"como `cs.umass.edu`. O **DNS** faz essa tradução entre nomes e endereços IP (e vice-versa), funcionando como um grande banco de dados distribuído, implementado em uma hierarquia de servidores de nomes, e também como um protocolo de camada de aplicação (usado por hosts e servidores DNS para se comunicarem).

O DNS **não poderia ser centralizado**: seria um ponto único de falha, teria um volume de tráfego impossível de suportar (só a Comcast recebe centenas de bilhões de consultas DNS por dia), e um banco de dados centralizado e distante aumentaria a latência. Por isso ele é organizado de forma hierárquica em três níveis principais:

- **Servidores raiz**: o contato de último recurso; existem 13 "servidores"lógicos de nomes raiz, replicados centenas de vezes pelo mundo.
- **Servidores de TLD (Top-Level Domain)**: responsáveis pelos domínios de nível superior, como `.com`, `.org`, `.edu` e os domínios de país (`.br`, `.uk`).
- **Servidores autoritativos**: mantidos pela própria organização (ou por seu provedor), fornecem o mapeamento definitivo de nome para IP dos hosts daquela organização.

Para resolver, por exemplo, `www.amazon.com`, o cliente primeiro consulta o servidor raiz (que indica o servidor `.com`), depois consulta o servidor `.com` (que indica o servidor da `amazon.com`) e, por fim, consulta o servidor da `amazon.com` para obter o endereço IP final. Essa consulta pode ser **iterada** (cada servidor devolve "não sei, pergunte a este outro servidor"para o cliente original) ou **recursiva** (cada servidor repassa a consulta adiante e só responde quando tiver a resposta final), sendo a iterada mais comum, pois distribui melhor a carga de processamento. Para melhorar o desempenho, os servidores fazem **cache** dos mapeamentos aprendidos, que expiram depois de um tempo (TTL).

O DNS armazena **registros de recursos (RR)**, no formato (nome, valor, tipo, ttl). Os principais tipos são: **A** (nome do host → endereço IP), **NS** (nome do domínio → servidor de nomes autoritativo daquele domínio), **CNAME** (um apelido para um nome canônico "verdadeiro") e **MX** (nome → servidor de e-mail SMTP associado).

4 Capítulo 3 — Camada de Transporte

4.1 Multiplexação e demultiplexação

A camada de transporte usa números de **porta** para entregar os dados que chegam da rede ao processo correto dentro de um host (demultiplexação) e para juntar dados de múltiplos processos em segmentos a serem enviados pela rede (multiplexação). Isso é o que permite que, no mesmo host, vários processos (um navegador, um cliente de e-mail, etc.) usem a rede "ao mesmo tempo", sem interferência entre eles.

4.2 UDP: o protocolo simples

O **UDP** (User Datagram Protocol) é um protocolo "sem frescuras": os segmentos podem se perder ou chegar fora de ordem, não há handshaking nem confirmação. Sua vantagem é justamente a simplicidade e a ausência de configuração prévia (sem o RTT gasto no handshake), o que o torna atraente para aplicações sensíveis a atraso.

Ainda assim, o UDP conta com um mecanismo simples de detecção de erros: a **soma de verificação** (*checksum*). A ideia é somar o conteúdo do segmento (tratado como uma sequência de números) e enviar o resultado junto; o receptor refaz a soma e verifica se bate com o valor recebido.

Exemplo simples de checksum: se a mensagem é composta pelos números 5 e 6, a soma enviada é 11. Se, por erro de transmissão, o receptor recebe os números 4 e 6, a soma calculada no destino também daria 10, diferente dos 11 recebidos no campo de checksum, e o erro seria detectado.

4.3 Transferência confiável de dados (rdt): a ideia por trás do TCP

Antes de estudar o TCP propriamente, a disciplina trabalha os **princípios** de como construir um protocolo de transferência confiável de dados (*reliable data transfer*, ou rdt) sobre um canal que pode cometer erros, de forma incremental:

- **rdt 1.0**: transferência confiável sobre um canal perfeitamente confiável (sem erros nem perdas) — não precisa de nenhum mecanismo extra.
- **rdt 2.0**: o canal pode ter erros de bit. A solução é usar **checksum** para detectar o erro e mensagens de reconhecimento: **ACK** (confirmação positiva, "recebi certo") e **NAK** (confirmação negativa, "recebi com erro, reenvie").
- **rdt 3.0**: além de erros de bit, o canal também pode **perder** pacotes inteiros. A solução é adicionar um **temporizador** (*timer*): se o remetente não recebe o ACK dentro de um tempo esperado, ele simplesmente retransmite o pacote, assumindo que ele (ou o ACK) se perdeu.

Esses protocolos de "um pacote por vez"são bastante ineficientes em redes com alto atraso, por isso o TCP de fato usa **pipelining** (enviar vários pacotes antes de receber a confirmação do primeiro), com técnicas como *Go-Back-N* e *Selective Repeat* para lidar com erros e perdas dentro desse fluxo contínuo.

4.4 TCP: transporte confiável orientado à conexão

O TCP estabelece uma conexão antes de transmitir dados, por meio do famoso **handshake de três vias** (*three-way handshake*):

1. O cliente envia um segmento **SYN** (com um número de sequência inicial x) para o servidor.
2. O servidor responde com um **SYNACK** (confirmando $x+1$ e enviando seu próprio número de sequência inicial y).
3. O cliente confirma com um **ACK** (confirmando $y+1$), e a conexão está estabelecida.

Durante a transmissão, o TCP calcula um **RTT estimado** de forma dinâmica, usando uma média móvel ponderada exponencial (EWMA) sobre as amostras de RTT reais:

$$EstimatedRTT = (1 - \alpha) \times EstimatedRTT + \alpha \times SampleRTT$$

E usa esse valor (junto com a variação do RTT, o $DevRTT$) para calcular o intervalo de *timeout* apropriado para a retransmissão:

$$TimeoutInterval = EstimatedRTT + 4 \times DevRTT$$

4.5 Controle de congestionamento TCP

O controle de congestionamento é o mecanismo pelo qual o remetente TCP ajusta a quantidade de dados que envia (a **janela de congestionamento**, ou *cwnd*) de acordo com o nível de congestionamento percebido na rede, mesmo sem ter uma visão global da rede. O algoritmo clássico (TCP Reno/Tahoe) segue o princípio **AIMD** (*Additive Increase, Multiplicative Decrease*: aumento aditivo, diminuição multiplicativa):

- **Aumento aditivo**: enquanto não há sinal de perda, a janela cresce de forma linear (soma 1 MSS por RTT), sondando a rede em busca de mais capacidade disponível de forma cautelosa.
- **Diminuição multiplicativa**: ao detectar uma perda (sinal de congestionamento), a janela é cortada pela metade, recuando rapidamente para aliviar a rede.

Esse comportamento de ”serra”(subindo devagar, caindo rápido) é conhecido como o **dente de serra** do AIMD. Antes disso, há uma fase inicial chamada **partida lenta** (*slow start*), em que a janela começa em 1 MSS e **dobra** a cada RTT (crescimento exponencial, mais rápido que o aumento aditivo), até atingir um limiar (threshold) — ponto em que o TCP passa a crescer de forma mais cautelosa (aditiva), para evitar sobrecarregar a rede de uma vez.

5 Capítulo 4 — Camada de Rede: Plano de Dados

5.1 Formato do datagrama IP

O datagrama IP carrega, entre outros, os campos: **versão** do protocolo IP, **comprimento do cabeçalho**, **comprimento total** do datagrama, um **identificador** e campos de **fragmentação** (usados quando o datagrama precisa ser dividido para caber em um link com MTU menor), o **TTL** (*Time To Live*: o número máximo de saltos restantes, decrementado a cada roteador, descartando o pacote quando chega a zero — evita que pacotes fiquem circulando indefinidamente em loops de roteamento), o **protocolo da camada superior** (indicando se o conteúdo é TCP ou UDP), além dos **endereços IP de origem e destino** (32 bits cada).

5.2 Endereçamento IP, sub-redes e CIDR

Um **endereço IP** é um identificador de 32 bits associado a cada **interface** de um host ou roteador (não ao dispositivo inteiro — um roteador, por exemplo, tem uma interface, e portanto um endereço IP, para cada link ao qual está conectado). Uma **sub-rede** é um conjunto de interfaces que conseguem se alcançar fisicamente sem passar por nenhum roteador intermediário; dentro de uma sub-rede, todos os endereços compartilham os mesmos bits de ordem alta (a "parte de rede"), diferindo apenas nos bits de ordem baixa restantes (a "parte de host").

O **CIDR** (Classless InterDomain Routing) é a notação usada para representar isso: `a.b.c.d/x`, em que x é o número de bits que formam a parte de sub-rede do endereço. Por exemplo, em `200.23.16.0/24`, os 24 bits mais à esquerda identificam a sub-rede, e os 8 bits restantes identificam os hosts dentro dela (permitindo até $2^8 = 256$ endereços nessa sub-rede).

Regra prática para calcular sub-redes: o tamanho do bloco de endereços é sempre uma potência de 2, calculada por $2^{(32-x)}$, onde x é a máscara. Em todo bloco de endereços: o **primeiro** endereço é o **endereço de rede** (identifica a sub-rede, não é atribuído a nenhum host), o **último** é o **endereço de broadcast** (usado para enviar uma mensagem a todos os hosts da sub-rede) e os endereços entre eles ficam disponíveis para os hosts. Por isso, a menor sub-rede útil (com pelo menos 2 endereços de host) precisa de um bloco de 4 endereços (máscara `/30`).

Exemplo: imagine que precisamos de uma sub-rede para uma rede com pelo menos 30 hosts válidos. Como $2^5 = 32$ (e descontando 2 endereços reservados — rede e broadcast — sobram 30 hosts válidos, exatamente o suficiente), usaríamos um bloco de 32 endereços, ou seja, máscara `/27` (pois $32 - 27 = 5$, e $2^5 = 32$).

O **DHCP** (Dynamic Host Configuration Protocol) é o protocolo que permite que um host obtenha dinamicamente (de forma "plug-and-play", sem configuração manual) seu endereço IP, a máscara de sub-rede, o endereço do roteador padrão e o endereço do servidor DNS local, a partir de um servidor DHCP presente na rede.

5.3 NAT: tradução de endereços de rede

O **NAT** (Network Address Translation) permite que todos os dispositivos de uma rede local compartilhem um único endereço IP público perante o restante da Internet, usando endereços privados internamente (faixas reservadas como `10.0.0.0/8`, `172.16.0.0/12` e `192.168.0.0/16`). O roteador NAT mantém uma **tabela de tradução**, associando o

par (endereço IP privado, porta de origem) ao par (endereço IP público do NAT, nova porta), reescrevendo os datagramas que saem e que entram da rede local de acordo com essa tabela.

As vantagens do NAT incluem: economizar endereços IP públicos (apenas um é necessário por toda a rede local), permitir trocar o ISP ou renumerar a rede interna sem afetar o mundo externo, e um benefício colateral de segurança (os dispositivos internos não são diretamente endereçáveis a partir de fora). Apesar de ser controverso tecnicamente (viola o princípio "fim a fim"da arquitetura original da Internet), o NAT é extremamente usado na prática, inclusive em redes celulares.

5.4 IPv6

A principal motivação do **IPv6** é o esgotamento do espaço de endereços do IPv4 (32 bits, cerca de 4,3 bilhões de endereços — a ICANN já alocou toda a última parcela de endereços IPv4 em 2011). O IPv6 usa endereços de **128 bits**, oferecendo um espaço praticamente inesgotável de endereços. A transição do IPv4 para o IPv6 ainda está em curso e não é trivial, já que os dois protocolos não são diretamente compatíveis entre si, exigindo mecanismos de transição (como tunelamento).

6 Capítulo 5 — Camada de Rede: Plano de Controle

Enquanto o Capítulo 4 cuidou do ”plano de dados”(como um datagrama individual é encaminhado por um roteador), este capítulo cuida do ”plano de controle”: como os roteadores descobrem, coletivamente, quais são os melhores caminhos pela rede.

6.1 Algoritmos de roteamento

Existem duas grandes famílias de algoritmos de roteamento:

- **Estado de enlace (link-state)**: cada nó conhece a topologia completa da rede e o custo de todos os enlaces (essa informação é obtida por ”transmissão do estado do enlace”a todos os outros nós), calculando de forma centralizada o caminho de menor custo. O algoritmo clássico dessa família é o **algoritmo de Dijkstra**.
- **Vetor de distância (distance-vector)**: cada nó conhece apenas o custo até seus vizinhos diretos, e troca periodicamente com eles suas estimativas de distância até os demais destinos, até que o processo convirja (baseado na equação de Bellman-Ford).

6.1.1 Algoritmo de Dijkstra: exemplo resolvido

O algoritmo de Dijkstra calcula, a partir de um nó de origem u, o caminho de menor custo até todos os outros nós da rede. Ele mantém um conjunto N' de nós cujo caminho de menor custo já é definitivamente conhecido e, a cada passo, adiciona a esse conjunto o nó mais próximo ainda não incluído, atualizando as estimativas de distância dos seus vizinhos.

Considerando uma rede com os nós u, v, w, x, y, z e os seguintes custos de enlace: $c_{u,v} = 2$, $c_{u,x} = 1$, $c_{u,w} = 5$, $c_{x,v} = 2$, $c_{x,w} = 3$, $c_{x,y} = 1$, $c_{v,w} = 3$, $c_{w,y} = 1$, $c_{w,z} = 5$, $c_{y,z} = 2$, o algoritmo evolui assim, partindo de u:

- **Passo 0**: $N' = \{u\}$. Inicializamos com os vizinhos diretos de u: $D(v) = 2$, $D(x) = 1$, $D(w) = 5$, $D(y) = \infty$, $D(z) = \infty$.
- **Passo 1**: o menor valor não confirmado é $D(x) = 1$, então x entra em N'. Atualizamos os vizinhos de x: $D(v) = \min(2, 1 + 2) = 2$, $D(w) = \min(5, 1 + 3) = 4$, $D(y) = \min(\infty, 1 + 1) = 2$.
- **Passo 2**: o menor valor é $D(y) = 2$, então y entra em N'. Atualizamos: $D(w) = \min(4, 2 + 1) = 3$, $D(z) = \min(\infty, 2 + 2) = 4$.
- **Passo 3**: o menor valor é $D(v) = 2$, então v entra em N'. Atualizamos: $D(w) = \min(3, 2 + 3) = 3$ (não muda).
- **Passo 4**: o menor valor é $D(w) = 3$, então w entra em N'. Atualizamos: $D(z) = \min(4, 3 + 5) = 4$ (não muda).
- **Passo 5**: resta apenas $D(z) = 4$, que entra em N', encerrando o algoritmo.

Ao final, a tabela de encaminhamento resultante em u fica: para o destino v, usa-se o enlace (u, v) diretamente; para todos os demais destinos (x, y, w, z), usa-se o enlace (u, x), já que a rota de menor custo para eles passa por x.

6.2 Roteamento na Internet real: OSPF e BGP

Na prática, a Internet não usa um único algoritmo de roteamento global (seria inviável em escala); ela é dividida em **Sistemas Autônomos (AS)**, cada um correspondendo, por exemplo, a um ISP. Dentro de um mesmo AS, usa-se um protocolo de roteamento *intra*-AS, sendo o **OSPF** (Open Shortest Path First, baseado em estado de enlace e no algoritmo de Dijkstra) o mais comum. Entre os diferentes AS, usa-se o protocolo **BGP** (Border Gateway Protocol), que permite que os ISPs anunciem (e escolham, de acordo com suas próprias políticas comerciais) quais rotas usar para alcançar outras redes, sendo o "cola"que efetivamente une a "rede de redes".

6.3 Plano de controle SDN

O **SDN** (*Software-Defined Networking*, ou rede definida por software) é uma abordagem mais moderna, em que o plano de controle é separado dos roteadores físicos e centralizado em um **controlador SDN** via software, que programa diretamente as tabelas de encaminhamento dos dispositivos de rede, dando mais flexibilidade e facilidade de gerenciamento à rede. O **ICMP** (Internet Control Message Protocol) também é citado nesse plano de controle, sendo usado para troca de mensagens de erro e sinalização entre roteadores (é, por exemplo, o protocolo usado pela ferramenta `traceroute` e pelo comando `ping`).

7 Capítulo 6 — A Camada de Enlace

7.1 Serviços da camada de enlace

A camada de enlace é responsável por transferir um datagrama entre nós fisicamente adjacentes (vizinhos diretos) através de um único link (com ou sem fio). Entre os serviços que ela pode oferecer estão o enquadramento (organizar os bits em **quadros**), controle de acesso ao enlace (quando o meio é compartilhado) e, principalmente, **detecção de erros**.

7.2 Detecção de erros: paridade e CRC

O método mais simples é a **verificação de paridade**: adiciona-se um bit extra de forma que o número total de bits "1"seja sempre par (ou ímpar, dependendo da convenção). Isso detecta um número ímpar de erros de bit, mas não consegue detectar (nem corrigir) todos os casos — por isso existe também a **paridade bidimensional**, que organiza os dados em uma matriz com bits de paridade por linha e por coluna, permitindo detectar *e* corrigir um único erro de bit (sem precisar retransmitir), pois o bit errado fica identificado pelo cruzamento da linha e da coluna com paridade incorreta.

Uma técnica mais robusta, usada de fato na prática (Ethernet, WiFi), é a **Verificação de Redundância Cíclica (CRC)**: o remetente escolhe r bits extras (os bits CRC, R) de forma que, tratando os dados D e o resultado $\langle D, R\rangle$ como um grande número binário, esse número seja exatamente divisível (em aritmética módulo 2, usando XOR) por um padrão de bits G acordado previamente entre as partes (o "gerador", com $r+1$ bits). O receptor repete a divisão de $\langle D, R\rangle$ por G: se o resto não for zero, um erro foi detectado. O CRC consegue detectar todas as rajadas de erro (*burst errors*) menores que $r+1$ bits, sendo bem mais robusto que a simples paridade.

7.3 Protocolos de acesso múltiplo

Quando vários nós compartilham o mesmo canal de transmissão (um **canal de acesso múltiplo**), pode ocorrer uma **colisão** quando dois ou mais nós transmitem ao mesmo tempo. Um protocolo de acesso múltiplo é um algoritmo distribuído que decide como e quando cada nó pode transmitir, sem nenhum canal "fora de banda"disponível para coordenar isso (a coordenação precisa, ela mesma, usar o canal compartilhado). Existem três grandes famílias:

- **Particionamento de canal**: divide o canal em "pedaços"menores dedicados a cada nó, seja no tempo (**TDMA** — Time Division Multiple Access, cada estação usa um slot de tempo fixo por rodada) ou na frequência (**FDMA** — Frequency Division Multiple Access, cada estação usa uma faixa de frequência fixa). O ponto fraco é que slots ou bandas não utilizados ficam ociosos, mesmo que outro nó tivesse dados para enviar naquele momento.

- **Acesso aleatório**: o canal não é dividido, e colisões são permitidas (e tratadas depois). Exemplos: ALOHA (e sua variante ALOHA com slot, em que o tempo é dividido em slots e, se houver colisão, cada nó retransmite com uma certa probabilidade em slots subsequentes) e o **CSMA/CD** (Carrier Sense Multiple Access with Collision Detection), usado classicamente na Ethernet com fio: o nó "escuta"o canal antes de transmitir (*carrier sense*) e, se detectar uma colisão durante a transmissão, para imediatamente de transmitir, reduzindo o tempo desperdiçado.

- **Revezamento ("passagem de bastão")**: os nós se revezam para transmitir, de forma ordenada, mas de forma flexível (quem tem mais dados pode usar o canal por mais tempo).

7.4 Endereçamento: MAC e ARP

Todo adaptador de rede tem um **endereço MAC** (*Media Access Control*), um identificador de 48 bits gravado fisicamente na placa de rede (diferente do endereço IP, que é lógico e pode mudar). Para que um host descubra o endereço MAC correspondente a um determinado endereço IP dentro da mesma sub-rede (já que, para enviar um quadro, é preciso saber o MAC de destino, não só o IP), usa-se o **ARP** (Address Resolution Protocol): o host envia uma solicitação ARP em *broadcast* perguntando "quem tem este endereço IP?", e o dono daquele IP responde diretamente com seu endereço MAC. Essa informação fica guardada temporariamente em uma **tabela ARP** local, evitando repetir a consulta a cada novo envio.

7.5 Ethernet

A **Ethernet** é, disparadamente, a tecnologia de LAN com fio mais usada no mundo. O **quadro Ethernet** é composto por: um **preâmbulo** (7 bytes de `10101010` seguidos de um byte `10101011`, usado para sincronizar o relógio do receptor com o do remetente), o **endereço de destino** e o **endereço de origem** (MAC, 6 bytes cada), um campo de **tipo** (indicando o protocolo da camada superior encapsulado), os **dados** (a carga útil, ou seja, o datagrama da camada de rede) e, por fim, o **CRC** para detecção de erros.

7.6 Switches: encaminhamento e autoaprendizagem

Um **switch** (comutador) é um dispositivo da camada de enlace que conecta vários hosts, encaminhando quadros de forma inteligente (olhando o endereço MAC de destino), ao contrário de um hub (que apenas repete o sinal para todas as portas). Cada link de um host até o switch é seu próprio domínio de colisão isolado (não há colisões dentro do switch), e múltiplas transmissões podem ocorrer simultaneamente entre pares diferentes de interfaces.

Os switches são **plug-and-play**: eles não precisam de configuração manual, graças ao processo de **autoaprendizagem**. A tabela de comutação começa vazia; quando um quadro chega, o switch "aprende"a localização do remetente, guardando na tabela o par (endereço MAC, interface de entrada, registro de tempo). Quando precisa encaminhar um quadro, o switch consulta essa tabela usando o endereço MAC de **destino**: se encontrar uma entrada, encaminha apenas pela interface correspondente; se não encontrar (ou for um endereço de broadcast), encaminha por todas as interfaces, exceto aquela por onde o quadro chegou (um processo chamado *flooding*).

8 Referências

Referências

[1] GOOGLE DRIVE. Redes de Computadores. Disponível em: `https://drive.google.com/drive/folders/1V-rtgEtJKDGxD-h_GhL4HSARQMetksuZ?usp=drive_link`. Acesso em: 22 out. 2026.

[2] KUROSE, J. F.; ROSS, K. W. *Redes de computadores e a Internet: uma abordagem top-down*. 8. ed. São Paulo: Pearson, 2020.

[3] FOROUZAN, B. A.; MOSHARRAF, F. *Redes de computadores: uma abordagem top-down*. Porto Alegre: AMGH, 2013.

[4] COMER, D. E. *Redes de computadores e internet*. 6. ed. Porto Alegre: Bookman, 2016.

[5] TANENBAUM, A. S.; WETHERALL, D. *Redes de computadores*. São Paulo: Pearson Prentice Hall, 2011.

[6] PETERSON, L. L.; DAVIE, B. S. *Redes de computadores: uma abordagem de sistemas*. Rio de Janeiro: Elsevier, 2013.